top-level actors or even to a shutdown of the `ActorSystem`. In the following sections, you learn the coding details of the actor lifecycle and supervision strategies.

5.2 Actor lifecycle events: Signals

An actor as an individual entity is spawned, lives for a while, and eventually stops. In the previous chapters, you have seen examples of how actors start or are initialized. If an actor contains a `Behavior.setup,` its code is executed immediately after spawning. This is its first event in life.

During an actor's lifetime, failures can trigger a restart before the actor finally stops. Regardless of whether an actor is forced to restart or stop due to a failure, it receives a message beforehand: a warning that one of these two actions is imminent. This is where the `ActorSystem` comes into play, sending these cautionary messages called *signals*. Depending on whether the strategy is a restart or a stop, the system sends a `PreRestart` or `PostStop` signal.

In Akka Typed, an actor not only understands its protocol (its explicit type) but also can deal with signal-type messages. They are an essential part of the `ActorSystem`, as they provide a way to deal with failures in the lifecycle of the actor and its children. From now on, let's distinguish between messages and signals. *Messages* refer to the protocol you've seen so far: command/event, request/response. On the other hand, a *signal* refers only to objects of the type `Signal`.